

FRONTEND ENGINEERING HANDOFF

Task Manager

Frontend Build Documentation

A step-by-step walkthrough of the React + Vite frontend: project setup, folder structure, and every file's code explained piece by piece.

React 19 · Vite 8 · React Router 7 · Axios · Bootstrap 5

Scope: frontend only — backend delivered separately

Contents

1. Project Setup
2. Folder Structure
3. src/main.jsx — Entry point — mounts the React app into the DOM
4. src/App.jsx — Root component — defines every route in the app
5. src/AuthContext.jsx — Global authentication state (React Context)
6. src/api.js — Shared Axios instance — every network call goes through this
7. src/components/ProtectedRoute.jsx — Route guard — blocks pages from unauthenticated users
8. src/components/Navbar.jsx — Top navigation bar shown on the Tasks page
9. src/pages/Login.jsx — Login screen
10. src/pages/Signup.jsx — Signup screen
11. src/pages/Tasks.jsx — Main screen — task list with search, filter, add/edit modal

Setup

Project Setup

Before any component code is written, scaffold the project and install its dependencies.

STEP 1 — CREATE THE PROJECT FOLDER WITH VITE

Vite's React template gives us a working React app in seconds — dev server, JSX support, and a build pipeline, with no manual Webpack/Babel config.

Terminal

Copy

```
npm create vite@latest frontend -- --template react
cd frontend
npm install
```

STEP 2 — INSTALL THE EXTRA LIBRARIES THIS APP USES

`axios` for HTTP calls to the backend API, and `react-router-dom` for client-side page routing (login/signup/tasks without full page reloads).

Terminal

Copy

```
npm install axios react-router-dom
```

STEP 3 — ADD BOOTSTRAP (STYLING)

No component library is used — just Bootstrap 5 loaded from a CDN link, added to the `<head>` of `index.html`. This keeps the setup simple: no extra npm package, no build step for CSS.

HTML

Copy

```
<link
  rel="stylesheet"
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
/>
```

STEP 4 — POINT THE FRONTEND AT THE BACKEND API

Create a `.env` file in the project root. Vite exposes any variable prefixed with `VITE_` to the client code via `import.meta.env`.

Terminal

Copy

```
VITE_API_URL=http://localhost:5000/api
```

STEP 5 — RUN IT

Terminal

 Copy

```
npm run dev
```

Layout

Folder Structure

Two folders under `src/`: `pages/` for full screens that get routed to, and `components/` for smaller reusable pieces shared across pages. Everything else (auth state, the API client, the router setup) lives directly in `src/`.

Text

Copy

```
frontend/
├─ index.html          (Bootstrap CDN link + <div id="root">)
├─ .env                (VITE_API_URL)
├─ package.json
├─ vite.config.js
├─ src/
│  ├─ main.jsx          entry point – mounts <App/>
│  ├─ App.jsx           route definitions
│  ├─ AuthContext.jsx   global auth state
│  ├─ api.js            shared axios instance
│  ├─ index.css
│  ├─ components/
│  │  ├─ Navbar.jsx
│  │  └─ ProtectedRoute.jsx
│  └─ pages/
│     ├─ Login.jsx
│     ├─ Signup.jsx
│     └─ Tasks.jsx
```

src/main.jsx

Entry point — mounts the React app into the DOM

Everything starts here. Vite loads this file first (wired up in `index.html`), and it's responsible for finding the empty `<div id="root">` and rendering our whole app into it, wrapped in the router.

PART 1 — IMPORTS

`StrictMode` is a React dev-mode helper that highlights unsafe patterns. `createRoot` is the React 18/19 API for mounting an app. `BrowserRouter` from `react-router-dom` enables client-side URL routing. We also pull in our global stylesheet and the root `App` component.

JavaScript

Copy

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import "./index.css";
import App from "./App.jsx";
```

PART 2 — MOUNT THE APP

`document.getElementById("root")` grabs the empty div from `index.html`. We render `<App />` inside `BrowserRouter` so every page can use routing hooks, and inside `StrictMode` for extra dev warnings.

JSX

Copy

```
createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

COMPLETE FILE (COPY THIS ONCE EVERY PART ABOVE MAKES SENSE)

src/main.jsx

 Copy

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import "./index.css";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);
```

src/App.jsx

Root component — defines every route in the app

This is the single place that decides which page shows for which URL, and whether the navbar should be visible.

PART 1 — IMPORTS

We bring in routing primitives (`Routes` , `Route` , `useLocation`), the `AuthProvider` that will wrap the whole app, the shared `Navbar` and `ProtectedRoute` components, and the three pages we route to.

JavaScript

Copy

```
import { Routes, Route, useLocation } from "react-router-dom";
import { AuthProvider } from "../AuthContext";
import Navbar from "../components/Navbar";
import ProtectedRoute from "../components/ProtectedRoute";
import Login from "../pages/Login";
import Signup from "../pages/Signup";
import Tasks from "../pages/Tasks";
```

PART 2 — HIDE THE NAVBAR ON AUTH PAGES

`useLocation()` tells us the current URL path. We don't want a nav bar with a logout button on the login/signup screens, so we compute a boolean flag for that.

JSX

Copy

```
export default function App() {
  const { pathname } = useLocation();
  const hideNavbar = ["/login", "/signup"].includes(pathname);
```

PART 3 — PROVIDER + ROUTES

`AuthProvider` wraps everything so any page can read the logged-in user. Inside, we conditionally render the navbar, then declare three routes. The `/` route (the Tasks page) is wrapped in `ProtectedRoute` so it redirects to `/login` if nobody is signed in.

JSX

 Copy

```
return (  
  <AuthProvider>  
    {!hideNavbar && <Navbar />}  
  
    <Routes>  
      <Route path="/login" element={<Login />} />  
      <Route path="/signup" element={<Signup />} />  
  
      <Route  
        path="/"  
        element={  
          <ProtectedRoute>  
            <Tasks />  
          </ProtectedRoute>  
        }  
      />  
    </Routes>  
  </AuthProvider>  
>;  
)
```

COMPLETE FILE (COPY THIS ONCE EVERY PART ABOVE MAKES SENSE)

src/App.jsx

 Copy

```
import { Routes, Route, useLocation } from "react-router-dom";
import { AuthProvider } from "../AuthContext";
import Navbar from "../components/Navbar";
import ProtectedRoute from "../components/ProtectedRoute";
import Login from "../pages/Login";
import Signup from "../pages/Signup";
import Tasks from "../pages/Tasks";

export default function App() {
  const { pathname } = useLocation();
  const hideNavbar = ["/login", "/signup"].includes(pathname);

  return (
    <AuthProvider>
      {!hideNavbar && <Navbar />}

      <Routes>
        <Route path="/login" element={<Login />} />
        <Route path="/signup" element={<Signup />} />

        <Route
          path="/"
          element={
            <ProtectedRoute>
              <Tasks />
            </ProtectedRoute>
          }
        />
      </Routes>
    </AuthProvider>
  );
}
```

src/AuthContext.jsx

Global authentication state (React Context)

Instead of passing the logged-in user down through props, we use React's Context API so any component can call `useAuth()` and get the user, plus `login()/logout()` functions.

PART 1 — IMPORTS + CREATE THE CONTEXT

`createContext(null)` creates a context object. `null` is the default value used only if a component tries to read it outside of a provider.

JavaScript

Copy

```
import { createContext, useContext, useState } from "react";

const AuthContext = createContext(null);
```

PART 2 — PROVIDER + INITIAL STATE

The initializer function passed to `useState` only runs once, on first render. It checks `localStorage` for a previously saved user so that refreshing the page doesn't log you out.

JSX

Copy

```
export function AuthProvider({ children }) {
  const [user, setUser] = useState(() => {
    const saved = localStorage.getItem("user");
    return saved ? JSON.parse(saved) : null;
  });
```

PART 3 — LOGIN()

Called after a successful API login. Persists both the user object and the JWT token to `localStorage`, then updates state so the whole app re-renders as "logged in".

JavaScript

Copy

```
const login = (userData, token) => {
  localStorage.setItem("user", JSON.stringify(userData));
  localStorage.setItem("token", token);
  setUser(userData);
};
```

PART 4 — LOGOUT()

Clears everything from storage and resets state, which sends the user back to the login screen via `ProtectedRoute` .

JavaScript

 Copy

```
const logout = () => {  
  localStorage.removeItem("user");  
  localStorage.removeItem("token");  
  setUser(null);  
};
```

PART 5 — PROVIDER RETURN + THE USEAUTH HOOK

We expose `{ user, login, logout }` to every child. `useAuth()` is just a shortcut so components don't need to import `useContext` and `AuthContext` everywhere.

JSX

 Copy

```
return (  
  <AuthContext.Provider value={{ user, login, logout }}>  
    {children}  
  </AuthContext.Provider>  
)  
};  
  
export const useAuth = () => useContext(AuthContext);
```

COMPLETE FILE (COPY THIS ONCE EVERY PART ABOVE MAKES SENSE)

src/AuthContext.jsx

 Copy

```
import { createContext, useContext, useState } from "react";

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(() => {
    const saved = localStorage.getItem("user");
    return saved ? JSON.parse(saved) : null;
  });

  const login = (userData, token) => {
    localStorage.setItem("user", JSON.stringify(userData));
    localStorage.setItem("token", token);
    setUser(userData);
  };

  const logout = () => {
    localStorage.removeItem("user");
    localStorage.removeItem("token");
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => useContext(AuthContext);
```

src/api.js

Shared Axios instance — every network call goes through this

Rather than importing axios directly in every page and repeating the base URL and auth header, we configure it once here.

PART 1 — CREATE THE INSTANCE

`axios.create()` makes a pre-configured axios instance. The base URL comes from the `VITE_API_URL` environment variable (set in `.env`), falling back to localhost for local dev.

JavaScript

Copy

```
import axios from "axios";

const API = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "http://localhost:5000/api",
});
```

PART 2 — REQUEST INTERCEPTOR

This function runs before every outgoing request. It reads the JWT saved at login time and attaches it as a `Bearer` token, so we never have to manually set auth headers on individual API calls.

JavaScript

Copy

```
API.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export default API;
```

COMPLETE FILE (COPY THIS ONCE EVERY PART ABOVE MAKES SENSE)

src/api.js

 Copy

```
import axios from "axios";

// One shared axios instance for the whole app.
const API = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "http://localhost:5000/api",
});

// Attach the saved token to every request automatically.
API.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export default API;
```

src/components/ProtectedRoute.jsx

Route guard — blocks pages from unauthenticated users

A tiny wrapper component. Anything passed in as `children` only renders if a user is logged in; otherwise it redirects to `/login`.

PART 1 — IMPORTS

`Navigate` is react-router's declarative redirect component. `useAuth` gives us the current user.

JavaScript

Copy

```
import { Navigate } from "react-router-dom";
import { useAuth } from "../AuthContext";
```

PART 2 — THE GUARD LOGIC

One line: if `user` exists, render whatever was wrapped (`children`); otherwise render a redirect to `/login` .

JSX

Copy

```
export default function ProtectedRoute({ children }) {
  const { user } = useAuth();
  return user ? children : <Navigate to="/login" />;
}
```

COMPLETE FILE (COPY THIS ONCE EVERY PART ABOVE MAKES SENSE)

src/components/ProtectedRoute.jsx

Copy

```
import { Navigate } from "react-router-dom";
import { useAuth } from "../AuthContext";

export default function ProtectedRoute({ children }) {
  const { user } = useAuth();
  return user ? children : <Navigate to="/login" />;
}
```

src/components/Navbar.jsx

Top navigation bar shown on the Tasks page

Displays the app name, and when a user is logged in, shows a greeting and a logout button.

PART 1 — IMPORTS

`Link` for client-side navigation without a full page reload, `useNavigate` to redirect programmatically after logout, and `useAuth` for the user + logout function.

JavaScript

 Copy

```
import { Link, useNavigate } from "react-router-dom";
import { useAuth } from "../AuthContext";
```

PART 2 — COMPONENT SETUP + LOGOUT HANDLER

Pulls `user` and `logout` from context. `handleLogout` clears auth state, then pushes the user to `/login`.

JSX

 Copy

```
export default function Navbar() {
  const { user, logout } = useAuth();
  const navigate = useNavigate();

  const handleLogout = () => {
    logout();
    navigate("/login");
  };
}
```

PART 3 — JSX

A Bootstrap dark navbar. The greeting and logout button are only rendered `{user && ...}` — i.e. only when someone is actually logged in.

JSX

 Copy

```
return (  
  <nav className="navbar navbar-dark bg-dark px-4 mb-4">  
    <Link to="/" className="navbar-brand fw-bold">  
      Task Manager  
    </Link>  
    {user && (  
      <div className="d-flex align-items-center gap-3">  
        <span className="text-white">Hi, {user.name}</span>  
        <button className="btn btn-outline-light btn-sm" onClick={handleLogout}>  
          Logout  
        </button>  
      </div>  
    )}  
  </nav>  
);  
}
```

COMPLETE FILE (COPY THIS ONCE EVERY PART ABOVE MAKES SENSE)

src/components/Navbar.jsx

 Copy

```
import { Link, useNavigate } from "react-router-dom";
import { useAuth } from "../AuthContext";

export default function Navbar() {
  const { user, logout } = useAuth();
  const navigate = useNavigate();

  const handleLogout = () => {
    logout();
    navigate("/login");
  };

  return (
    <nav className="navbar navbar-dark bg-dark px-4 mb-4">
      <Link to="/" className="navbar-brand fw-bold">
        Task Manager
      </Link>
      {user && (
        <div className="d-flex align-items-center gap-3">
          <span className="text-white">Hi, {user.name}</span>
          <button className="btn btn-outline-light btn-sm" onClick={handleLogout}>
            Logout
          </button>
        </div>
      )}
    </nav>
  );
}
```

src/pages/Login.jsx

Login screen

A two-column screen: a branded panel on the left, a login form on the right. On submit it calls the backend, stores the session via AuthContext, and redirects home.

PART 1 — IMPORTS

`useState` for form fields, `Link` / `useNavigate` from the router, our shared `API` instance, and `useAuth` to save the session after login.

JavaScript

Copy

```
import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import API from "../api";
import { useAuth } from "../AuthContext";
```

PART 2 — STATE + CHANGE HANDLER

One state object holds both fields. `change` is a generic handler — it reads `e.target.name` so the same function works for every input, keyed by each input's `name` attribute.

JSX

Copy

```
export default function Login() {
  const [form, setForm] = useState({ email: "", password: "" });
  const [error, setError] = useState("");
  const { login } = useAuth();
  const navigate = useNavigate();

  const change = e => setForm({ ...form, [e.target.name]: e.target.value });
```

PART 3 — SUBMIT HANDLER

Prevents the default form POST, calls `POST /auth/login`, and on success calls context's `login()` with the returned user + token, then navigates to `/`. On failure it reads the backend's error message and shows it.

JSX

Copy

```
const submit = async e => {
  e.preventDefault();
  setError("");
  try {
    const { data } = await API.post("/auth/login", form);
    login(data.user, data.token);
    navigate("/");
  } catch (e) {
    setError(e.response?.data?.message || "Login failed");
  }
};
```

PART 4 — JSX: BRANDED LEFT PANEL

Purely visual — a gradient panel with the product pitch. Hidden on small screens (`d-none d-md-flex`).

JSX

Copy

```
return (
  <div className="min-vh-100 d-flex">
    <div className="w-50 d-none d-md-flex flex-column justify-content-center p-5 text-white"
      style={{ background: "linear-gradient(135deg,#071d52,#0866e9)" }}>
      <h2 className="fw-bold mb-5">✓ Task Manager</h2>
      <h1 className="fw-bold display-5">
        Organize tasks.<br />Boost productivity.<br />Achieve more.
      </h1>
      <p className="fs-5 mt-3 opacity-75">
        Manage your tasks, deadlines and projects all in one place.
      </p>
    </div>
```

PART 5 — JSX: THE FORM PANEL

Controlled inputs bound to `form.email` / `form.password`, an error alert that only shows when `error` is set, and a link over to the signup page.

JSX

Copy

```
<div className="flex-grow-1 d-flex align-items-center justify-content-center bg-
light p-4">
  <div className="bg-white p-5 rounded-4 shadow" style={{ width: "100%",
maxWidth: 480 }}>
    <div className="text-center mb-4">
      <h2 className="fw-bold mt-2">Welcome Back</h2>
      <p className="text-muted">Sign in to continue to your account</p>
    </div>

    {error && <div className="alert alert-danger">{error}</div>}

    <form onSubmit={submit}>
      <label className="form-label">Email</label>
      <input className="form-control form-control-lg mb-3" type="email"
        name="email" placeholder="Enter your email"
        value={form.email} onChange={change} required />

      <label className="form-label">Password</label>
      <input className="form-control form-control-lg mb-4" type="password"
        name="password" placeholder="Enter your password"
        value={form.password} onChange={change} required />

      <button className="btn btn-primary btn-lg w-100">Login</button>
    </form>

    <p className="text-center mt-4 mb-0">
      Don't have an account? <Link to="/signup">Sign up</Link>
    </p>
  </div>
</div>
);
}
```

COMPLETE FILE (COPY THIS ONCE EVERY PART ABOVE MAKES SENSE)

src/pages/Login.jsx

Copy

```
import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import API from "../api";
import { useAuth } from "../AuthContext";

export default function Login() {
  const [form, setForm] = useState({ email: "", password: "" });
  const [error, setError] = useState("");
  const { login } = useAuth();
  const navigate = useNavigate();

  const change = e => setForm({ ...form, [e.target.name]: e.target.value });

  const submit = async e => {
    e.preventDefault();
    setError("");
    try {
      const { data } = await API.post("/auth/login", form);
      login(data.user, data.token);
      navigate("/");
    } catch (e) {
      setError(e.response?.data?.message || "Login failed");
    }
  };

  return (
    <div className="min-vh-100 d-flex">
      <div className="w-50 d-none d-md-flex flex-column justify-content-center p-5 text-white"
        style={{ background: "linear-gradient(135deg,#071d52,#0866e9)" }}>
        <h2 className="fw-bold mb-5">✓ Task Manager</h2>
        <h1 className="fw-bold display-5">
          Organize tasks.<br />Boost productivity.<br />Achieve more.
        </h1>
        <p className="fs-5 mt-3 opacity-75">
          Manage your tasks, deadlines and projects all in one place.
        </p>
      </div>

      <div className="flex-grow-1 d-flex align-items-center justify-content-center bg-light p-4">
        <div className="bg-white p-5 rounded-4 shadow" style={{ width: "100%",
          maxWidth: 480 }}>
          <div className="text-center mb-4">
            <h2 className="fw-bold mt-2">Welcome Back</h2>
            <p className="text-muted">Sign in to continue to your account</p>
          </div>
        </div>
      </div>
    </div>
  );
}
```


src/pages/Signup.jsx

Signup screen

Same layout pattern as Login, but with more fields, and on success it redirects to `/login` instead of logging the user in directly.

PART 1 — IMPORTS

Notice there's no `useAuth` here — signup doesn't log the user in, it just creates the account and sends them to the login page.

JavaScript

Copy

```
import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import API from "../api";
```

PART 2 — STATE + CHANGE HANDLER

Four controlled fields this time: name, username, email, password.

JSX

Copy

```
export default function Signup() {
  const [form, setForm] = useState({ name: "", username: "", email: "", password: "" });
  const [error, setError] = useState("");
  const navigate = useNavigate();

  const change = e => setForm({ ...form, [e.target.name]: e.target.value });
```

PART 3 — SUBMIT HANDLER

Calls `POST /auth/signup`. On success there's no token to store — we just navigate to `/login` so the user signs in normally.

JSX

Copy

```
const submit = async e => {
  e.preventDefault();
  setError("");
  try {
    await API.post("/auth/signup", form);
    navigate("/login");
  } catch (e) {
    setError(e.response?.data?.message || "Signup failed");
  }
};
```

PART 4 — JSX

Same two-panel layout as Login, with the extra Name/Username fields and a link back to </login>.

JSX

Copy

```
return (
  <div className="min-vh-100 d-flex">
    <div className="w-50 d-none d-md-flex flex-column justify-content-center p-5 text-white"
      style={{ background: "linear-gradient(135deg,#071d52,#0866e9)" }}>
      <h2 className="fw-bold mb-5">Task Manager</h2>
      <h1 className="fw-bold display-5">
        Organize tasks.<br />Boost productivity.<br />Achieve more.
      </h1>
      <p className="fs-5 mt-3 opacity-75">
        Manage your tasks, deadlines and projects all in one place.
      </p>
    </div>

    <div className="flex-grow-1 d-flex align-items-center justify-content-center bg-light p-4">
      <div className="bg-white p-5 rounded-4 shadow" style={{ width: "100%",
        maxWidth: 480 }}>
        <div className="text-center mb-4">
          <h2 className="fw-bold mt-2">Create Account</h2>
          <p className="text-muted">Sign up to start managing your tasks</p>
        </div>

        {error && <div className="alert alert-danger">{error}</div>}

        <form onSubmit={submit}>
          <label className="form-label">Full Name</label>
          <input className="form-control form-control-lg mb-3"
            name="name" placeholder="Enter your full name"
            value={form.name} onChange={change} required />

          <label className="form-label">Username</label>
          <input className="form-control form-control-lg mb-3"
            name="username" placeholder="Choose a username"
            value={form.username} onChange={change} />

          <label className="form-label">Email</label>
          <input className="form-control form-control-lg mb-3" type="email"
            name="email" placeholder="Enter your email"
            value={form.email} onChange={change} required />

          <label className="form-label">Password</label>
          <input className="form-control form-control-lg mb-4" type="password"
            name="password" placeholder="Create a password"
            value={form.password} onChange={change} required />
        </form>
      </div>
    </div>
  </div>
)
```

```
        <button className="btn btn-primary btn-lg w-100">Sign Up</button>
    </form>

    <p className="text-center mt-4 mb-0">
        Already have an account? <Link to="/login">Login</Link>
    </p>
</div>
</div>
</div>
);
}
```

COMPLETE FILE (COPY THIS ONCE EVERY PART ABOVE MAKES SENSE)

src/pages/Signup.jsx

Copy

```
import { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import API from "../api";

export default function Signup() {
  const [form, setForm] = useState({ name: "", username: "", email: "", password: "" });
  const [error, setError] = useState("");
  const navigate = useNavigate();

  const change = e => setForm({ ...form, [e.target.name]: e.target.value });

  const submit = async e => {
    e.preventDefault();
    setError("");
    try {
      await API.post("/auth/signup", form);
      navigate("/login");
    } catch (e) {
      setError(e.response?.data?.message || "Signup failed");
    }
  };

  return (
    <div className="min-vh-100 d-flex">
      <div className="w-50 d-none d-md-flex flex-column justify-content-center p-5 text-white"
        style={{ background: "linear-gradient(135deg,#071d52,#0866e9)" }}>
        <h2 className="fw-bold mb-5">Task Manager</h2>
        <h1 className="fw-bold display-5">
          Organize tasks.<br />Boost productivity.<br />Achieve more.
        </h1>
        <p className="fs-5 mt-3 opacity-75">
          Manage your tasks, deadlines and projects all in one place.
        </p>
      </div>

      <div className="flex-grow-1 d-flex align-items-center justify-content-center bg-light p-4">
        <div className="bg-white p-5 rounded-4 shadow" style={{ width: "100%",
          maxWidth: 480 }}>
          <div className="text-center mb-4">
            <h2 className="fw-bold mt-2">Create Account</h2>
            <p className="text-muted">Sign up to start managing your tasks</p>
          </div>

          {error && <div className="alert alert-danger">{error}</div>}
        </div>
      </div>
    </div>
  );
}
```

```

    <form onSubmit={submit}>
      <label className="form-label">Full Name</label>
      <input className="form-control form-control-lg mb-3"
        name="name" placeholder="Enter your full name"
        value={form.name} onChange={change} required />

      <label className="form-label">Username</label>
      <input className="form-control form-control-lg mb-3"
        name="username" placeholder="Choose a username"
        value={form.username} onChange={change} />

      <label className="form-label">Email</label>
      <input className="form-control form-control-lg mb-3" type="email"
        name="email" placeholder="Enter your email"
        value={form.email} onChange={change} required />

      <label className="form-label">Password</label>
      <input className="form-control form-control-lg mb-4" type="password"
        name="password" placeholder="Create a password"
        value={form.password} onChange={change} required />

      <button className="btn btn-primary btn-lg w-100">Sign Up</button>
    </form>

    <p className="text-center mt-4 mb-0">
      Already have an account? <Link to="/login">Login</Link>
    </p>
  </div>
</div>
</div>
);
}

```

src/pages/Tasks.jsx

Main screen — task list with search, filter, add/edit modal

The core feature of the app. This one file handles fetching tasks, searching/filtering them client-side, and creating/editing/deleting/cycling status through a single modal + table.

PART 1 — IMPORTS + LOOKUP CONSTANTS

`P` / `S` translate the backend's numeric `priority` / `status` enums into readable labels. `C` maps priority to a Bootstrap badge color. `empty` is the blank template used to reset the add/edit form.

JavaScript

Copy

```
import { useEffect, useState } from "react";
import API from "../api";

const P = ["Low", "Medium", "High"], S = ["Not Started", "In Progress", "Completed"];
const C = ["success", "warning", "danger"], empty = { title: "", description: "", priority: 1 };
};
```

PART 2 — STATE + INITIAL LOAD

`tasks` holds the list from the server; `form` / `editId` / `modal` drive the add/edit dialog; `search` / `filter` drive client-side filtering. `load()` fetches all tasks for the logged-in user, and `useEffect` runs it once on mount.

JSX

Copy

```
export default function Tasks() {
  const [tasks, setTasks] = useState([]), [form, setForm] = useState(empty);
  const [editId, setEditId] = useState(null), [modal, setModal] = useState(false);
  const [search, setSearch] = useState(""), [filter, setFilter] = useState("");

  const load = async () => setTasks((await API.get("/tasks/get_all_tasks")).data.tasks);
  useEffect(() => { load(); }, []);
}
```

PART 3 — CHANGE() AND OPEN()

`change` is the same generic form handler pattern used on Login/Signup. `open(t)` is called both by "+ Add Task" (no argument → blank form) and by "Edit" (passes the task → pre-fills the form and remembers its id in `editId`).

JSX

Copy

```
const change = e => setForm({ ...form, [e.target.name]: e.target.value });
const open = (t = null) => {
  setEditId(t?._id || null);
  setForm(t ? { title: t.title, description: t.description, priority: t.priority } :
empty);
  setModal(true);
};
```

PART 4 — SAVE(), REMOVE(), STATUS()

`save` branches on whether `editId` is set: PUT to update, or POST to create. `remove` confirms before calling DELETE. `status` is the quick one-click status cyler on the badge — it increments status mod 3 (Not Started → In Progress → Completed → back to Not Started). All three re-run `load()` afterward to refresh the table.

JSX

Copy

```
const save = async e => {
  e.preventDefault();
  editId
    ? await API.put(`/tasks/update_task/${editId}`, form)
    : await API.post("/tasks/create_task", form);
  setModal(false); load();
};

const remove = async id => {
  if (!confirm("Delete this task?")) return;
  await API.delete(`/tasks/delete_task/${id}`); load();
};

const status = async t => {
  await API.put(`/tasks/update_task/${t._id}`, { status: (t.status + 1) % 3 });
  load();
};
```

PART 5 — FILTERED LIST

This is computed on every render, not stored in state — it filters the full `tasks` array by the search text (case-insensitive title match) and the selected priority.

JavaScript

Copy

```
const list = tasks.filter(t =>
  t.title.toLowerCase().includes(search.toLowerCase()) &&
  (filter === "" || t.priority == filter)
);
```

PART 6 — JSX: TOOLBAR (SEARCH, FILTER, ADD BUTTON)

Three Bootstrap columns: a search input, a priority `<select>` built from the `P` array, and the button that opens the modal in "add" mode.

JSX

Copy

```
return (
  <div className="container py-5" style={{ maxWidth: 1400 }}>
    <h2 className="fw-bold">My Tasks</h2>
    <p className="text-muted">View and manage your tasks in one place.</p>

    <div className="bg-white border rounded-4 shadow-sm p-4">
      <div className="row g-3 mb-4">
        <div className="col-md-5">
          <input className="form-control" placeholder=" Search tasks..."
            onChange={e => setSearch(e.target.value)} />
        </div>
        <div className="col-md-4">
          <select className="form-select" onChange={e =>
setFilter(e.target.value)}>
            <option value="">All Priorities</option>
            {P.map((x, i) => <option key={x} value={i}>{x}</option>)}
          </select>
        </div>
        <div className="col-md-3">
          <button className="btn btn-primary w-100" onClick={() => open()}>
            + Add Task
          </button>
        </div>
      </div>
    </div>
  </div>
```

PART 7 — JSX: THE TABLE

Maps over the filtered `list`. The status cell is itself a button — clicking it calls `status(t)` to cycle to the next state. Edit/Delete buttons call `open(t)` and `remove(t._id)`.

JSX

Copy

```
<div className="table-responsive">
  <table className="table align-middle">
    <thead className="table-light">
      <tr><th>Title</th><th>Description</th><th>Priority</th>
<th>Status</th><th>Actions</th></tr>
    </thead>
    <tbody>
      {list.map(t => (
        <tr key={t._id}>
          <td className="fw-semibold">{t.title}</td>
          <td className="text-muted">{t.description || "-"}</td>
          <td><span className={`badge bg-${C[t.priority]}`}>
{P[t.priority]}</span></td>
          <td><button className="badge bg-secondary border-0"
onClick={() => status(t)}>{S[t.status]}</button></td>
          <td>
            <button className="btn btn-sm btn-outline-primary me-
2" onClick={() => open(t)}>Edit</button>
            <button className="btn btn-sm btn-outline-danger"
onClick={() => remove(t._id)}>Delete</button>
          </td>
        </tr>
      )})}
    </tbody>
  </table>
</div>
</div>
```

PART 8 — JSX: THE ADD/EDIT MODAL

Only rendered when `modal` is true. The heading and submit button label switch based on `editId` ("Add New Task" vs "Edit Task"). It's a real `<form>` so pressing Enter submits it via `save`.

JSX

Copy

```
{modal && <>
  <div className="modal d-block" style={{ background: "#0006" }}>
    <div className="modal-dialog modal-dialog-centered">
      <form className="modal-content border-0 shadow" onSubmit={save}>
        <div className="modal-header">
          <h5 className="modal-title">{editId ? "Edit Task" : "Add New
Task"}</h5>
          <button type="button" className="btn-close" onClick={() =>
setModal(false)} />
        </div>
        <div className="modal-body">
          <input required name="title" className="form-control mb-3"
placeholder="Task title" value={form.title} onChange=
{change} />
          <textarea name="description" className="form-control mb-3"
rows="3"
placeholder="Description" value={form.description}
onChange={change} />
          <select name="priority" className="form-select"
value={form.priority} onChange={change}>
            {P.map((x, i) => <option key={x} value={i}>{x}</option>)}
          </select>
        </div>
        <div className="modal-footer">
          <button type="button" className="btn btn-light" onClick={()
=> setModal(false)}>Cancel</button>
          <button className="btn btn-primary">{editId ? "Update Task" :
"Add Task"}</button>
        </div>
      </form>
    </div>
  </div>
</>
</div>
);
}
```

COMPLETE FILE (COPY THIS ONCE EVERY PART ABOVE MAKES SENSE)

src/pages/Tasks.jsx

 Copy

```
import { useEffect, useState } from "react";
import API from "../api";

const P = ["Low", "Medium", "High"], S = ["Not Started", "In Progress", "Completed"];
const C = ["success", "warning", "danger"], empty = { title: "", description: "", priority: 1 };

export default function Tasks() {
  const [tasks, setTasks] = useState([]), [form, setForm] = useState(empty);
  const [editId, setEditId] = useState(null), [modal, setModal] = useState(false);
  const [search, setSearch] = useState(""), [filter, setFilter] = useState("");

  const load = async () => setTasks((await API.get("/tasks/get_all_tasks")).data.tasks);
  useEffect(() => { load(); }, []);

  const change = e => setForm({ ...form, [e.target.name]: e.target.value });
  const open = (t = null) => {
    setEditId(t?._id || null);
    setForm(t ? { title: t.title, description: t.description, priority: t.priority } :
empty);
    setModal(true);
  };

  const save = async e => {
    e.preventDefault();
    editId
      ? await API.put(`/tasks/update_task/${editId}`, form)
      : await API.post("/tasks/create_task", form);
    setModal(false); load();
  };

  const remove = async id => {
    if (!confirm("Delete this task?")) return;
    await API.delete(`/tasks/delete_task/${id}`); load();
  };

  const status = async t => {
    await API.put(`/tasks/update_task/${t._id}`, { status: (t.status + 1) % 3 });
    load();
  };

  const list = tasks.filter(t =>
    t.title.toLowerCase().includes(search.toLowerCase()) &&
    (filter === "" || t.priority == filter)
  );

  return (
```

```

<div className="container py-5" style={{ maxWidth: 1400 }}>
  <h2 className="fw-bold">My Tasks</h2>
  <p className="text-muted">View and manage your tasks in one place.</p>

  <div className="bg-white border rounded-4 shadow-sm p-4">
    <div className="row g-3 mb-4">
      <div className="col-md-5">
        <input className="form-control" placeholder=" Search tasks..."
          onChange={e => setSearch(e.target.value)} />
      </div>
      <div className="col-md-4">
        <select className="form-select" onChange={e =>
setFilter(e.target.value)}>
          <option value="">All Priorities</option>
          {P.map((x, i) => <option key={x} value={i}>{x}</option>)}
        </select>
      </div>
      <div className="col-md-3">
        <button className="btn btn-primary w-100" onClick={() => open()}>
          + Add Task
        </button>
      </div>
    </div>

    <div className="table-responsive">
      <table className="table align-middle">
        <thead className="table-light">
          <tr><th>Title</th><th>Description</th><th>Priority</th>
<th>Status</th><th>Actions</th></tr>
        </thead>
        <tbody>
          {list.map(t => (
            <tr key={t._id}>
              <td className="fw-semibold">{t.title}</td>
              <td className="text-muted">{t.description || "-"}</td>
              <td><span className={`badge bg-${C[t.priority]}`}>
{P[t.priority]}</span></td>
              <td><button className="badge bg-secondary border-0"
                onClick={() => status(t)}>{S[t.status]}</button></td>
              <td>
                <button className="btn btn-sm btn-outline-primary me-
2" onClick={() => open(t)}>Edit</button>
                <button className="btn btn-sm btn-outline-danger"
onClick={() => remove(t._id)}>Delete</button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  </div>
</div>

```

```

    {modal && <>
      <div className="modal d-block" style={{ background: "#0006" }}>
        <div className="modal-dialog modal-dialog-centered">
          <form className="modal-content border-0 shadow" onSubmit={save}>
            <div className="modal-header">
              <h5 className="modal-title">{editId ? "Edit Task" : "Add New
Task"}</h5>
              <button type="button" className="btn-close" onClick={() =>
setModal(false)} />
            </div>
            <div className="modal-body">
              <input required name="title" className="form-control mb-3"
placeholder="Task title" value={form.title} onChange=
{change} />
              <textarea name="description" className="form-control mb-3"
rows="3"
placeholder="Description" value={form.description}
onChange={change} />
              <select name="priority" className="form-select"
value={form.priority} onChange={change}>
                {P.map((x, i) => <option key={x} value={i}>{x}</option>)}
              </select>
            </div>
            <div className="modal-footer">
              <button type="button" className="btn btn-light" onClick={()
=> setModal(false)}>Cancel</button>
              <button className="btn btn-primary">{editId ? "Update Task" :
"Add Task"}</button>
            </div>
          </form>
        </div>
      </div>
    </>
  </div>
);
}

```